BIOISO: Biological Isomorphisms in Formal Self-Maintaining Systems

BIOISO: Biological Isomorphisms in Formal Self-Maintaining Systems

Author: Juan Carlos Ghiringhelli (Pragmaworks)
Status: Preprint v1 — operational model, structural analogy, and empirical validation of the T5 primitive (BBOB §4.5, AEGIS §4.6). The title names the research program; the *formal* autopoietic isomorphism with Maturana–Varela autopoiesis is the subject of a forthcoming companion paper currently in preparation. What this iteration delivers vs. what is deferred is documented explicitly in §1.1 (Scope of claims) and §3.5 (Relation to autopoiesis).
Repository: github.com/jghiringhelli/loom
Related: *Onwards: The Formal Tradition Was Waiting for Its Executor* (Ghiringhelli, 2026 — companion essay carrying the autopoietic argument); Loom Language Manual (`docs/manual.md` in the repository — the canonical reference for the language); Generative Specification (Ghiringhelli, 2026 — Zenodo DOI 10.5281/zenodo.19637142).

Abstract

We present BIOISO — a biologically-inspired framework for computational entities that adapt their own algorithmic structure across generations. A BIOISO entity is not a meta-heuristic; it is a being whose genome encodes an optimization strategy, whose meiosis mechanism promotes surviving mutations into the next compiled binary, and whose telomere lifecycle bounds the scope of structural change within a generation. We argue for a five-tier ceiling hierarchy (T1–T5), supported by proof sketches in §2 and empirical validation in §4.5–§4.6: each tier adds exactly one primitive that the tier below cannot express, and each primitive enables a class of convergence that is structurally unreachable without it. The T5 primitive — structural self-modification via meiosis — is the first mechanism that operates on the space of *algorithms*, not the space of parameter values or operator selections. We implement the hierarchy in the Loom language (`solver_tiers.rs`, `examples/ladder.loom`, `bench_colony_ladder`, and the `.loom`→BIOISO bridge in `being_loader.rs`). We empirically validate the T5 structural primitive in two settings: the COCO/BBOB f2 ill-conditioned ellipsoid (§4.5, 10× median NF reduction across 30 trials against a Halton-approximated T4 baseline — a public

benchmark controlled experiment, not itself a "domain" in §5's sense; the Halton substitution is a benchmark scope limitation discussed in §4.5) and the AEGIS delta-neutral DeFi strategy across five market regimes (§4.6, inter-generational topology switching with epoch-2 (StrongBull) per-epoch Sharpe advantage of +0.517, but **net 5-epoch cumulative Sharpe of −0.024 — a small loss, not a gain** — driven by parameter re-convergence cost at the return-Ranging boundary (−0.339) plus a MildBear false-positive (−0.285) outweighing the StrongBull win; per (StrongBull + return-Ranging) cycle the net is +0.178 Sharpe, so compounding cumulative advantage is projected over repeated such cycles in longer backtests, but is not demonstrated within the 5-epoch window). The BIOISO model seeds ten domains in §5: eight T5 domains (one empirically validated — AEGIS — and seven theoretically motivated, where the structural criterion for T5 is satisfied but empirical validation is forthcoming work), plus two calibration domains (T3 ocean_circulation, T4 biosphere) included to demonstrate that the framework correctly assigns lower tiers where they are sufficient and does not default to T5 everywhere. The framing of BIOISO as a *formal* biological isomorphism — in the sense of Maturana and Varela's autopoiesis (1972) — is deferred to a companion paper currently in preparation; this paper uses "biologically-inspired" to denote structural inspiration, not formal organizational-closure equivalence.

---

1. Introduction

The history of heuristic optimization is a history of ceilings. Greedy algorithms (T1) saturate on any instance requiring backtracking. Simulated annealing (T2) escapes local optima but saturates when the optimal operator class changes. Hyper-heuristics (T3) adapt operator selection but saturate when the operator portfolio is architecturally wrong. Bayesian optimization (T4) is sample-efficient but saturates when the surrogate model's architecture cannot represent the objective surface.

> **A note on tier namespaces.** The T1–T5 labels used throughout this paper refer to the **optimization algorithm hierarchy** defined in §2 (greedy → simulated annealing → hyper-heuristic → Bayesian → meiosis). The Generative Specification (GS) system-architecture framework defines a *different* tier hierarchy — T1–T6 deployment tiers (development → staging → production → evolution → synthesis → meta-telos) — described in Ghiringhelli (2026). The numbering overlaps but the namespaces are distinct: a "T4" in this paper is a Bayesian optimization algorithm; a "T4" in the GS framework is the BIOISO / evolution deployment tier (which itself uses the algorithm hierarchy below as building blocks). Readers coming to this paper from the *Onwards!* essay or other GS material should keep the two namespaces separate when interpreting tier references.

Each ceiling is not a failure of implementation — it is a structural property of the tier's expressive power. A T1 algorithm cannot discover that its greedy rule is wrong by applying the rule more carefully. A T2 algorithm cannot invent a new neighbourhood operator by exploring more of the existing space. The ceiling is *built into the tier's combinatorial geometry*.

BIOISO is the answer to the question: what is the tier above T4? The answer is not "a better surrogate model" or "a smarter acquisition function." Those are T4 improvements. The answer is a tier whose adaptations operate on the space of algorithms — where the entity can structurally replace which algorithm it runs, and where that replacement persists across generations through a meiosis mechanism that compiles the surviving mutation into the next binary.

This paper makes four contributions:

1. **Ceiling hierarchy argument (Section 2):** A proof-sketch-structured argument that T1–T5 form a strict hierarchy, with each tier's ceiling explicitly derived from its primitive's expressive bounds. The arguments are sketches in the academic sense — they identify the structural reason a ceiling exists and cite the underlying impossibility result (e.g., No Free Lunch, RKHS-closure); they are not machine-checked proofs.

2. **BIOISO operational model (Section 3):** A specification of the T5 entity — genome, telomere lifecycle, meiosis mechanism, and the conditions under which structural mutation fires.

3. **Implementation in Loom (Section 4):** The full keyword-level implementation of T1–T5 in the Loom language, including `plasticity:`, `learn:`, `rewire:`, and the runtime `solver_tier` auto-escalation mechanism. Two controlled experiments validate the T5 structural primitive: §4.5 on the COCO/BBOB benchmark suite, §4.6 on the AEGIS DeFi strategy.

4. **Domain seeding (Section 5):** Ten domains seeded under the BIOISO model. Empirical status: **one T5 domain empirically validated** (`aegis_delta_neutral`, §4.6) **plus one separate controlled primitive experiment** (BBOB f2, §4.5, validating the T5 *primitive* on a public benchmark not itself a §5 "domain"); seven T5 domains theoretically motivated (§5.1–§5.7); two calibration domains (§5.9 biosphere T4, §5.10 ocean_circulation T3) included to demonstrate the framework does not default to T5 universally. The seven motivated T5 domains satisfy the structural criterion for T5 (the optimal solution class changes during the experiment horizon); empirical validation on each is forthcoming work.

1.1 Scope of claims

To prevent the framing from outrunning the evidence, this paper distinguishes four levels of claim and notes which apply where:

- **Proved:** Properties whose argument follows from a cited published result applied to the construction here — e.g., the T1 ceiling follows from No Free Lunch (Wolpert & Macready 1997) applied to a non-stationary instance distribution; the T4 ceiling follows from RKHS-closure properties of Gaussian-process kernels.
- **Argued (proof sketch):** Properties 1 (Monotone Lineage) and 2 (Structural Escape) in §3.4 are presented as arguments with named scope limitations. Property 1 establishes a per-mutation drift-improvement invariant that lifts to a lineage-level expectation under stated assumptions; the full lineage-level monotonicity in non-stationary regimes is not proved here. Property 2 is an existence argument conditional on candidate-pool adequacy.

- **Empirically validated:** The T5 structural primitive on BBOB f2 (10× median NF reduction across 30 trials, §4.5 — a public benchmark, not a "domain") and on the AEGIS bimodal regime structure (10/10 correct topology switches at StrongBull epochs, §4.6 — the `aegis_delta_neutral` T5 domain, §5.8).
- **Theoretically motivated, empirically unvalidated:** Seven of the eight T5 domains in §5 — namely §5.1 amr_coevolution, §5.2 flash_crash, §5.3 adaptive_jit, §5.4 protein_drug_resistance, §5.5 ics_zero_day, §5.6 quantum_error_mitigation, §5.7 climate_intervention. The structural criterion for T5 is satisfied for each; the T5 mechanism is specified; their empirical validation analogous to §4.6 is forthcoming work.
- **Calibration:** §5.9 biosphere (T4) and §5.10 ocean_circulation (T3) are deliberately *not* BIOISO/T5 entities. They are included to demonstrate that the framework correctly assigns lower tiers where they are sufficient, providing the falsifiability boundary for the "T5 is necessary" claim.

What this paper does **not** claim: - A formal autopoietic isomorphism (Maturana & Varela 1972) between BIOISO meiosis and biological self-production. The structural inspiration is acknowledged; the formal organizational-closure equivalence is the subject of a companion paper in preparation (see §3.5). - Deployment-validated performance for any of the seven theoretically-motivated T5 domains (§5.1–§5.7). - Convergence rate bounds on the meiosis cycle. Property 2 is an existence argument, not a bound.

The broader *speculative argument* — that BIOISO and Loom exhibit structural analogy with biological self-maintaining systems, and that this analogy is a load-bearing piece of a larger thesis about *directed formal autopoiesis* — is developed in the companion essay *Onwards: The Formal Tradition Was Waiting for Its Executor* (Ghiringhelli 2026, submitted to ACM SIGPLAN Onward! 2026). That essay frames the analogy in structural terms consistent with §3.5 of this paper; the *formal* organizational-closure equivalence (autopoiesis in the Maturana–Varela sense) is deferred to forthcoming work in both documents. The present paper supplies the operational model and initial empirical evidence; the essay supplies the theoretical framing; neither claims formal isomorphism as established.

---

2. The Five-Tier Ceiling Hierarchy

> **Tier-naming note (BIOISO vs GS lifecycle cascade).** The BIOISO tier hierarchy named in §2.1–§2.6 enumerates *optimization complexity* — fixed-rule dispatch (T1) through structural self-modification via meiosis (T5), mapping the algorithmic primitives that can or cannot escape a saturation regime. This is distinct from the GS obligation cascade developed in the GS white paper (Ghiringhelli 2026, §4.1.f) and summarised in the companion essay [Ghiringhelli 2026, *Onwards*, §V], which enumerates *software-lifecycle stages* in a **six-tier cascade**: T1 Development, T2 Staging / Pre-prod, T3 Production, T4 Evolution, T5 Synthesis (architecturally specified), T6 Meta-telos (research agenda). The harness is *not a tier* in the GS framework — it is a cross-cutting capability that recurs at every tier with stage-appropriate tests; the judgment layer is also *not a tier* — it is the irreducible human work

> that no future tier removes. The BIOISO and GS namespaces overlap by accident — both use Tn — and converge at one point: **GS T4 (the *Evolution* lifecycle stage) is the lifecycle stage at which BIOISO operates**; **BIOISO T5 (the *structural self-modification* primitive) is the operational mechanism the GS T4 stage employs**. When this paper writes "T5" without qualifier, BIOISO T5 is meant; when the GS lifecycle stage is meant, the paper writes "GS T4 (Evolution)" or another explicit qualifier. This namespace separation is deliberate: BIOISO is a domain-specific instantiation of the GS T4 stage, not a parallel framework.

2.1 Definitions

Definition 1 (Optimization tier): A tier is characterized by the primitive it adds to the tier below. A being at tier T possesses all primitives of tiers 1 through T.

Definition 2 (Saturation): A tier-T being *saturates* on problem instance P if there exists k such that for all n > k, the being's improvement in objective value per iteration is zero — the drift score `D(t)` plateaus above the telos threshold for all subsequent ticks.

Definition 3 (Structural escape): A tier-(T+1) being *structurally escapes* saturation on P if it can reduce drift below the telos threshold using only the primitive added at tier T+1 — the escape is not achievable by any being at tier T regardless of parameter settings.

2.2 Tier 1: Fixed-Rule Dispatch

Primitive: `telos:` + `function:` — a pure function that maps state to action with no feedback.

Canonical algorithms: SPT (Smith 1956), DSATUR (Brélaz 1979), First-Fit Decreasing (Johnson 1974), Nearest Neighbor TSP.

T1 ceiling theorem: Any fixed dispatch rule saturates on any input class for which the rule was not designed. Formally: there exists a problem class P for which no fixed mapping `f: State → Action` produces a monotone improvement sequence over all instances in P.

Proof sketch: By the No Free Lunch theorem (Wolpert & Macready 1997), a fixed rule that is optimal over one distribution of instances is suboptimal over the complementary distribution. A non-stationary environment generates instances from a distribution that shifts over time, so any fixed rule eventually faces a majority of instances it was not designed for.

What T2 adds: The ability to accept worsening moves with probability `exp(-ΔE/T)`, escaping the local optimum that the greedy rule terminates at.

2.3 Tier 2: Stochastic Neighbourhood Search

Primitive: `evolve: simulated_annealing | genetic_algorithm | ils` — a stochastic search that explores the solution neighbourhood.

Canonical algorithms: Simulated Annealing (Kirkpatrick et al. 1983), Genetic Algorithm (Holland 1975), Iterated Local Search with Or-opt perturbation (Lourenço et al. 2003).

T2 ceiling theorem: A fixed-operator stochastic search saturates when the *class* of optimal move changes. Specifically, if the problem distribution shifts such that the neighbourhood defined by the current operator no longer contains improving solutions within polynomial reach, T2 cannot escape.

Proof sketch: The SA acceptance criterion allows uphill moves, but only within the operator's neighbourhood geometry. If the optimal solution requires a move that the operator cannot express (e.g., a cross-route exchange when the operator only performs within-route swaps), no temperature schedule allows SA to find it. The operator portfolio is fixed at compile time.

What T3 adds: The ability to select *which* operator to apply at each decision point, based on observed performance.

2.4 Tier 3: Adaptive Operator Selection (Hyper-Heuristic)

Primitive: `plasticity:` — a weight table that selects between heuristic operators using reinforcement learning.

Canonical algorithms: SARSA selection hyper-heuristic (Sutton & Barto 1998; Burke et al. 2013), Q-learning over operator space.

The key distinction from T2: the SARSA agent operates on the *space of operators*, not the space of solutions. Given operators `{H_1, H_2, H_3}` and state `s`, it selects `H_i` to apply next based on a learned weight table (Sutton & Barto 1998, §6.4). The operators themselves are fixed; the agent learns which one to use when.

T3 ceiling theorem: A hyper-heuristic saturates when the optimal operator is *not in the portfolio*. If the non-stationary environment requires an operator class `H_novel` that was not declared at compile time, no weight table update can produce `H_novel`.

Proof sketch: The SARSA update rule is `W[i] += α(r - W[i])` — it adjusts weights toward observed reward. If `max_i(W[i])` over all declared operators still produces drift above telos threshold, and the optimal action lies in a class not representable by any operator in the portfolio, no learning rate `α` allows convergence.

What T4 adds: A learned surrogate model over the objective surface, enabling the being to direct evaluations toward regions of likely improvement without requiring a pre-specified operator.

2.5 Tier 4: Surrogate-Model Optimization

Primitive: `learn: gaussian_process | attention_model` — a probabilistic model over the (configuration, objective) space.

Canonical algorithms: Bayesian Optimization with GP-UCB (Srinivas et al. 2010, Snoek et al. 2012), Attention Model with Pointer Network and REINFORCE training (Kool et al. 2019, Vinyals et al. 2015).

GP-UCB selects the next configuration by maximising `UCB(x) = μ(x) + β·σ(x)`, balancing exploitation (high μ) against exploration (high σ). The GP posterior updates after each observation, making future selections progressively more sample-efficient.

T4 ceiling theorem: A GP-based optimizer saturates when the optimal function lies outside its kernel's RKHS. Formally: there exists an objective `f*` such that for any GP with kernel `k`, the predictive posterior `μ_n(x)` does not converge to `f*(x)` as `n → ∞` if `f* ∉ H_k` (the reproducing kernel Hilbert space of `k`).

Proof sketch: The GP posterior mean is a weighted sum of kernel evaluations. If `f*` is not in the closure of this space (e.g., `f*` has a discontinuous structure that the RBF kernel cannot represent), the GP will not converge regardless of the number of observations. The kernel is fixed at compile time; changing it requires a new model architecture.

What T5 adds: The ability to structurally replace the optimization algorithm — not just its parameters — when saturation is detected.

2.6 Tier 5: Structural Self-Modification via Meiosis

Primitives: `rewire:` (intra-generational structural replacement) + `telomere:` (generational lifecycle boundary) + meiosis (inter-generational mutation promotion).

The T5 distinction: T1–T4 all operate on the *same* optimization problem with the *same* algorithm class. They differ in how much they adapt *within* the algorithm class. T5 operates on the space of algorithm classes — it can replace which algorithm it runs, and this replacement is compiled into the next generation's binary.

The escape mechanism: When `D_static > threshold` for `k` consecutive ticks (`rewire:` trigger), the BIOISO replaces its dispatch strategy from a candidate pool (`candidates: [...]`). The replacement is selected by `fitness_guided` scoring — the candidate that, when simulated on recent signal history, produces the greatest drift reduction. The meiosis loop then bakes the surviving replacement into the genome file for the next compiled binary.

---

3. The BIOISO Formal Model

3.1 Components

A BIOISO entity `B` is a tuple `(G, T, M, Ω)` where:

- **G (Genome):** A `.loom` specification file encoding the being's current algorithm tier, parameter bounds, and structural constraints. The genome is machine-readable and *GS-derivable*.[1] Derivability is the foundational GS property: every other artifact (Rust source, TypeScript declarations, OpenAPI definitions, JSON Schema) is mechanically derivable from the same `.loom` source; see Ghiringhelli (2026, Generative Specification) for the formal treatment.

- **T (Telomere):** A bounded counter `t ∈ [0, t_max]` incremented on each generational division. When `t = t_max`, the entity enters senescence: surviving mutations are committed to the genome and a new entity is spawned from the updated specification.

- **M (Mutation set):** The set of promoted mutations `{m_1, ..., m_n}` accumulated during the current generation. Each mutation `m_i` is one of: `ParameterAdjust(entity, param, δ)`, `StructuralRewire(entity, signal, target)`, `EntityClone(source, new_id)`, `EntityPrune(entity)`, or `EntityRollback(entity, checkpoint)`.

- **Ω (Orchestrator):** The runtime that dispatches T1–T4 proposal generators based on `live_params["solver_tier"]` and detects saturation conditions that trigger structural mutation.

3.2 The Generational Lifecycle

```
Generation n:
  Entity spawned from genome G_n
  Telomere counter t = 0
  live_params["solver_tier"] = baseline_tier(G_n)

  While t < t_max:
    Tick: receive signals, dispatch solver_tier, observe outcome
    If saturation(k consecutive same-direction promotions):
      If solver_tier < 4: escalate solver_tier (intra-generational
T1→T4)
      Else: fire rewire: (replace dispatch strategy)
      Append mutation to M
    t += 1

  Senescence:
    Filter M: keep mutations that improved drift
    Write surviving mutations to G_{n+1}
    Compile G_{n+1} → new binary
    Spawn entity from G_{n+1}
```

3.3 The Meiosis Gate

The meiosis gate is the *design specification* for which mutations are promoted from generation `n` to `n+1`. The gate is specified as three filters; the implementation status of each filter, made explicit below, varies.

1. **Improvement filter:** Mutation `m` is kept iff the drift score decreased by at least `δ_min` after `m` was applied. **Implementation status:** specified; partially enforced in the runtime `MeiosisEngine` (mutations that did not reduce drift at application time are rejected pre-workflow), but the workflow itself does not currently re-verify drift reduction at the genome boundary.

2. **Safety filter:** Mutation `m` is rejected if it would cause any safety annotation violation (`@bounded_telos`, `@corrigible`, `@mortal`, `@sandboxed`) in the compiled genome. **Implementation status:** specified; enforced indirectly by the loom checker (`SafetyChecker`) as compile-time errors when a mutation produces a genome that violates a safety annotation — but this is a *compile* check, not an explicit gate filter. Mutations that pass `cargo build` are assumed safe by the workflow; if the safety check ever ran as a separate stage it would catch the same set, so the current arrangement is sound but not what §3.3 strictly describes.

3. **Stability filter:** Mutation `m` is rejected if the cargo tests (`cargo test --lib`) fail after applying `m` in isolation. **Implementation status:** specified and enforced — `.github/workflows/evolve.yml` runs `cargo build` and `cargo test --lib` as the post-application gate. This is the filter the workflow directly implements.

Implementation-vs-specification scoping. The three-filter description above is the design specification. What `.github/workflows/evolve.yml` actually executes today is the stability filter (cargo build + cargo test –lib). The improvement and safety filters are enforced upstream of the workflow — the improvement filter inside the runtime `MeiosisEngine` (which proposes mutations only when drift improvement is observed at application time), the safety filter inside the loom compiler (`SafetyChecker` rejects builds that violate safety annotations). The pipeline is the *composition* of these three locations, not a single artifact. Bringing all three into a single explicit gate stage — re-verifying drift improvement *and* safety annotations *and* cargo tests at the workflow boundary — is forthcoming infrastructure work, listed in §8 Limitations. A reviewer inspecting `evolve.yml` will see only the stability stage; they should read §3.3 as the *spec* the full pipeline approximates, not as a literal description of what one file contains.

3.4 Formal Properties

Property 1 (Monotone lineage — proof sketch under stationarity): Let `D(G_n)` be the mean drift score across all ticks of generation `n`, and let `D̃(m, G_n)` denote the drift improvement at the tick when mutation `m` was applied during generation `n`. Under the assumption that the instance distribution at generations `n` and `n+1` is identically distributed (i.i.d. across generation boundaries), `E[D(G_{n+1})] ≤ E[D(G_n)]`.

Proof-sketch argument: The meiosis gate (§3.3) only promotes a mutation `m` if `D̃(m, G_n) ≥ δ_min > 0` — the mutation reduced drift at its application tick by at least `δ_min`. If no mutations pass the gate, `G_{n+1} = G_n` and `D(G_{n+1}) = D(G_n)` deterministically. If mutations are promoted, then on instances drawn from the same distribution as those of generation `n`, each promoted mutation produces a drift improvement bounded below by `δ_min` in expectation, so generation `n+1`'s expected mean drift is at most `D(G_n) − (Σ_m δ_min)/T_max`.

Scope limitations of this argument: - *Non-stationarity.* In genuinely non-stationary regimes (the case the framework is designed for, see §1), the instance distribution at generation `n+1` differs from generation `n`. The per-tick improvement at application time does not directly bound the next generation's expected drift over its full lifetime. A complete proof requires bounding the regret between application-time drift improvement and lifetime expected drift; this is beyond the present paper. - *Stability filter coverage.* The gate's stability filter (`cargo test --lib` passes after mutation) is necessary but not sufficient for downstream lineage monotonicity — passing tests at one generation does not guarantee non-degradation under a subsequent regime change. - *Adversarial mutation composition.* Two mutations each individually drift-reducing may interact non-additively in composition. The current gate evaluates mutations in isolation against the post-application state but does not jointly validate the full promoted set.

A complete proof of lineage-level monotonicity under non-stationary regimes is forthcoming work.

Property 2 (Structural escape): For any problem class `P` where T1–T4 saturate, there exists a sequence of structural mutations `m_1, ..., m_k` such that `D(G_0 + m_1 + ... + m_k) < τ_telos` (below telos threshold).

Proof sketch: The candidates pool in `rewire: candidates: [...]` can include `novel_hypothesis` — a placeholder for algorithm classes not present in the initial genome. The MeiosisEngine generates candidate implementations from the genome specification using the GS derivation rules. Since the genome is a Turing-complete specification (any algorithm expressible in the loom type system can be encoded), there exists a sequence of structural mutations that can represent any computable optimization strategy. *Note on scope:* This is an existence argument — it establishes that a convergent mutation sequence exists for any computable problem class, not that the meiosis engine will find it in a bounded number of generations. In practice, the candidate pool must include the target algorithm class for convergence to occur; curating the initial candidate pool is a human design decision.

What `novel_hypothesis` operationally is. Because the reviewer-facing meaning of the existence argument depends on what the MeiosisEngine actually does when it encounters `novel_hypothesis`, we describe it concretely. In the current implementation, `novel_hypothesis` triggers an LLM-mediated derivation step: the engine reads the genome's machine-readable `-- GS EVOLUTION SPEC` blocks plus the surrounding `telos:` and `metric_bounds:` declarations, and asks an LLM to propose a candidate algorithm class expressible in the existing loom type system. The proposal is then subjected to the three-filter meiosis gate (§3.3) — `cargo test --lib` must pass, no safety annotation may be violated, and the candidate must produce a measured drift reduction on the held-out signal history before it is promoted. This means

Property 2's existence argument is not a claim about deterministic formal derivation: it is the claim that *given* an LLM-mediated candidate generator and *given* an adequate test/safety/improvement gate, the search space of candidate algorithms is unrestricted (any computable optimization strategy is representable in the loom type system). The validity of the existence argument is therefore conditional on the candidate generator producing a viable proposal within the generation budget; the gate is the convergence filter, not the search itself. This is closer in spirit to AlphaDev-style RL search (Mankowitz et al. 2023, §6.6) than to deterministic program synthesis — with the important difference that the meiosis gate enforces both cross-generational persistence and safety-annotation invariance, both absent from AlphaDev. A deterministic-only (LLM-free) version of the engine, restricted to genome-template recombination over a fixed candidate pool, is a subset of the current implementation and is the case in which the existence argument reduces to a finite-pool search problem.

Property 3 (Bounded change): The structural change per generation is bounded by `|M_promoted|` — the number of promoted mutations. Since each mutation has a declared type and safety filter, the space of possible structural changes per generation is finite.

3.5 Relation to autopoiesis (scope of the biological-isomorphism claim)

The title of this paper uses "biologically-inspired." It does *not* use "biological isomorphism" in the strong formal sense that Maturana and Varela introduced. The distinction is load-bearing for what the paper claims.

What we mean by "biologically-inspired." The mechanisms named in §3.1 — genome, telomere, meiosis — are taken from biology as structural inspiration. The names denote the architectural role each plays (an information store that is read but never consumed; a generational lifecycle counter; a mutation-promotion gate operating on the information store), and the role is structurally similar to the biological mechanism that shares the name. This is a *functional analogy* — the resemblance is at the level of role and operation, not at the level of a proved equivalence between the formal systems.

What we do not claim. Maturana and Varela (1972) defined autopoiesis as the organizational closure of a self-producing system — a system whose components produce themselves and through that self-production maintain the system's boundary. Subsequent work (Di Paolo 2005; Bianchini 2023) formalizes the criteria under which a non-biological system can be said to exhibit autopoiesis: organizational closure, self-production of components, boundary maintenance, and a recognized substrate-independence argument. **Establishing that BIOISO satisfies these criteria is a separate piece of work that this paper does not attempt.** The architecture is *consistent* with autopoietic structure — the genome produces the binary that produces the next genome, the meiosis gate maintains the boundary of admissible mutations, the telomere defines a generational closure — but consistency is not equivalence. The formal demonstration requires biological-systems expertise this paper's author does not bring alone.

Forthcoming work. A companion paper, currently in preparation in collaboration with a biological-systems researcher, develops the formal autopoietic isomorphism: an organizational-closure equivalence between BIOISO's `(G, T, M, Ω)` structure and the Maturana–Varela definition, with explicit treatment of substrate-independence under Di Paolo's adaptivity criteria. That paper, when complete, will replace the present paper's "biologically-inspired" framing with the stronger formal claim where it is warranted, and identify where it is not.

Where the broader argument lives. The theoretical framing connecting BIOISO to the larger thesis — that the formal tradition of correct computing has been waiting for an executor (the LLM as Logos to the formal Nous), that GS is the discipline that connects them, and that the resulting systems exhibit *directed formal autopoiesis* as a structural consequence — is developed in the companion essay *Onwards: The Formal Tradition Was Waiting for Its Executor* (Ghiringhelli 2026, submitted to ACM SIGPLAN Onward! 2026). The essay carries the autopoietic argument; this paper supplies the operational model and the initial empirical evidence. Readers seeking the theoretical scaffolding should read the two together; readers seeking the proof of formal isomorphism should wait for the forthcoming companion paper.

---

4. Implementation in Loom

4.1 The `examples/ladder.loom` Specification

The canonical T1→T5 progression is specified in `examples/ladder.loom` as five `being:` blocks for the single-machine job scheduling problem:

```
T1SPTScheduler    — telos: + function: only
T2SAScheduler     — + evolve: simulated_annealing
T3HyperScheduler  — + plasticity: sarsa
T4BayesScheduler  — + learn: gaussian_process
T5BIOISOScheduler — + rewire: + telomere: (meiosis via GS genome
loop)
```

The file compiles clean (`loom compile examples/ladder.loom → OK`), demonstrating syntactic validity of the five-tier vocabulary — the loom checker accepts each `being:` block under its declared tier's required keyword set. This is a syntactic validation only; semantic adequacy (that each tier's runtime behavior matches the tier's theoretical description) is established by the experiments in §4.5 and §4.6.

4.2 `src/runtime/solver_tiers.rs`

The T1–T4 proposal generators are pure functions dispatched by the orchestrator based on `live_params["solver_tier"]`:

Function	Tier	Algorithm
`t1_greedy(event)`	T1	Fixed delta on worst metric

Function	Tier	Algorithm
`t2_sa(event, temp, rng)`	T2	Boltzmann: `p_uphill = exp(-score/T)`
`t3_sarsa(event, weights, ε, rng)`	T3	ε-greedy over `N_HEURISTICS = 3` operator types (Sutton & Barto 1998)
`t4_ucb_bandit(event, history, metrics, β, N)`	T4	Bandit UCB: `μ + β√(ln(N+1)/(n+1))` — *not* GP-UCB (see implementation note)

Auto-escalation in `orchestrator.rs`: when the same parameter + direction is promoted `2 × tier1_fail_threshold` consecutive times, `solver_tier` increments by 1.0 and the orchestrator logs `[tier_up] entity: T{n} → T{n+1} (saturation × k)`.

Implementation note — T4 is bandit UCB, not GP-UCB. §2.5 defines T4 as Gaussian-process or attention-model surrogate optimization (GP-UCB). The function the orchestrator dispatches as "T4" in `solver_tiers.rs` is a *bandit-style* UCB — mean improvement plus an exploration bonus of the form `β√(ln(N+1)/(n+1))` — without a Gaussian-process posterior or any kernel-based surrogate. This is a deliberate scope choice for the current implementation (no GP dependency, no kernel hyperparameter tuning) but it means the runtime T4 is functionally closer to a UCB-1 bandit than to GP-UCB. A full GP-UCB implementation is forthcoming work; until it lands, claims that the runtime "implements T4" should be read as "implements a bandit approximation of T4". The §2.5 ceiling argument — that GP-UCB saturates on objectives outside the kernel's RKHS — is preserved as a *conceptual* T4 ceiling; the *runtime* T4 has a different, weaker ceiling (the bandit UCB cannot model the objective surface at all, only the per-arm empirical reward distribution).

Implementation note — T3/T5 boundary blur, and the rewire signal. The T3 SARSA portfolio in `solver_tiers.rs` carries three heuristic slots: `H_SMALL_ADJUST`, `H_LARGE_ADJUST`, and `H_REWIRE`. The first two emit `MutationProposal::ParameterAdjust`; the third currently emits `MutationProposal::StructuralRewire`. Strictly, this blurs the conceptual T3/T5 boundary defined in §2.4 vs §2.6: by the hierarchy, a hyper-heuristic at T3 should select among operators within its portfolio, not emit structural rewires (those should be T5-exclusive). **The current code's design intent is that `H_REWIRE` functions as a *saturation signal*: when SARSA weights converge on the `H_REWIRE` slot, the orchestrator reads this as "T3 has exhausted its operator portfolio and structural change is needed" and escalates `live_params["solver_tier"]` toward T5.** The emitted `StructuralRewire` proposal is, in this reading, a signal-payload rather than a true intra-T3 structural emission. A clean refactor that separates the saturation *signal* (T3-emittable, telling the orchestrator to escalate) from the structural *emission* (T5-only, generated by `rewire:` rule firing) is forthcoming work; the conceptual hierarchy in §2 — that only T5 owns structural emission as a first-class primitive — is the load-bearing claim and is unchanged by the implementation boundary blur.

4.3 The `bench_colony_ladder` Proof

`src/bin/bench_colony_ladder.rs` runs five entities over 60 ticks on a non-stationary drift signal `D(t, ϕ) = 0.3|sin(0.05t + ϕ)| + 0.05ε`:

Entity	Starting tier	Final tier	Convergence
`scheduling_t1`	T1-Greedy	T1-Greedy	no (ceiling)
`scheduling_t2`	T2-SA	T2-SA	yes (slow)
`scheduling_t3`	T3-SARSA	T3-SARSA	yes
`scheduling_t4`	T4-GP-UCB	T4-GP-UCB	yes
`scheduling_t5_capable`	T1-Greedy	T2-SA	yes (auto-escalated at tick 6)

The `scheduling_t5_capable` entity (named *T5-capable* rather than *T5* because it carries the full T5 keyword set — `rewire:` + `telomere:` — but does not exercise inter-generational meiosis within the 60-tick budget) fires `[tier_up] T1 → T2` at tick 6 (saturation ×6), demonstrating the intra-generational auto-escalation mechanism. The final tier is T2-SA because the non-stationary drift does not persist long enough to saturate T2 within 60 ticks. The T5 inter-generational primitive itself is validated separately in §4.5 (BBOB controlled experiment) and §4.6 (AEGIS controlled experiment); this benchmark is a proof-of-mechanism for T1→T4 auto-escalation only.

Scope note: `bench_colony_ladder` demonstrates *intra-generational* auto-escalation (T1→T4) — the mechanism by which a single entity discovers a higher-tier algorithm during its current generation. It does not demonstrate *inter-generational meiosis*, where the genome file is rewritten and recompiled between generations. Cross-generational T5 meiosis operates via the `.github/workflows/evolve.yml` GS genome loop, which requires a full compile/test cycle between generations and is exercised in the `experiments/bioiso/` suite. The T5 structural primitive itself — basis-rotation `StructuralRewire` — is validated independently in §4.5 using the COCO/BBOB benchmark suite (Hansen et al. 2009).

4.4 The `.loom` → BIOISO Bridge (`being_loader.rs`)

The bridge module `src/runtime/being_loader.rs` closes the gap between the loom specification language and the BIOISO runtime. Any `.loom` file containing beings with `telos:` declarations can be loaded directly into the colony without manual wiring:

```
loom runtime load examples/apex_colony.loom --dry-run
# → detected 7 beings: BiogeochemicalEngine (T5), CarbonCycleReg
        (T4), ...

loom runtime load examples/apex_colony.loom --db bioiso.db
# → spawned 7 DynamicBIOISOSpec entities in bioiso.db
```

The bridge infers tier from declared features (Section 3), extracts metric bounds from `telos.bounded_by` clauses, and generates baseline signals from `matter:` field types. This means any `being:` specification written in loom is simultaneously:

1. A **typed loom program** (checker, codegen, tests)
2. A **BIOISO entity specification** (immediately loadable into the colony)
3. A **GS genome unit** (mutatable by the MeiosisEngine)

The three representations are derived from the same source file — there is no separate wiring, no translation layer, and no runtime-only configuration. The specification IS the colony entity.

4.5 BBOB Controlled Experiment — T5 Primitive Validation

To validate the T5 structural primitive in a controlled, reproducible setting, we run a two-condition experiment on four functions from the COCO/BBOB benchmark suite (Hansen et al. 2009): **f1 Sphere** (unimodal, symmetric), **f2 Separable Ellipsoid** (unimodal, condition number 10^6), **f15 Rastrigin** (multimodal, ~10^10 local optima at DIM=10), and **f24 Lunacek bi-Rastrigin** (bimodal basin structure).

Experimental design: 30 independent trials, 200 ticks, DIM=10. The **control** condition runs T1–T4 only (greedy coordinate descent → simulated annealing → SARSA-step adaptation → Halton quasi-random). The **experimental** condition adds T5: after 20 ticks of stagnation, a random orthogonal basis rotation (Gram-Schmidt on LCG vectors) is generated; it is accepted only if a 10-step probe strictly improves normalized fitness. Seeds are fixed per trial (seed i for trial i) — no cherry-picking. Full source and evidence: `src/runtime/bbob.rs`, `src/bin/bbob_experiment.rs`, `experiments/bbob/evidence/`.

T4 baseline scope limitation. §2.5 defines T4 as Gaussian-process or attention-model surrogate optimization (GP-UCB), but the T4 stage in this benchmark is implemented as Halton quasi-random space-filling — a deliberately lightweight approximation chosen to keep the experiment reproducible without a GP dependency and to isolate the T5 structural-rewire mechanism from confounders introduced by GP kernel selection and hyperparameter tuning. The practical consequence is that the T1–T4 control under-represents the strongest T4 instantiation a reviewer might expect: a full GP-UCB with an RBF kernel would model the f2 ellipsoidal valley and would likely achieve sample-efficiency competitive with — or superior to — the T1–T5 condition on this specific function. **The result in this section is therefore an upper bound on the T5 advantage over a Halton-T4 baseline, not over a GP-UCB-T4 baseline.** Likewise, the comparison does not include CMA-ES (Hansen & Ostermeier 2001), which learns a full covariance matrix that performs structurally similar basis rotation to the T5 primitive demonstrated here and would likely solve f2 without requiring an external structural-rewire mechanism. We acknowledge both gaps explicitly: a full GP-UCB and a CMA-ES comparison on f2 are forthcoming work. The present experiment establishes that the T5 *primitive* — random orthogonal basis rotation with a

fitness-gated accept rule — is selective and load-bearing on the f2 landscape; it does not establish that T5 is the *only* mechanism capable of solving f2, nor that T5 out-performs the strongest existing rotation-aware T2-class method.

Convergence rate (% of 30 trials reaching NF ≤ 0.01):

Function	Multimodal	T1–T4	T1–T5	Δ	T1–T4 Med tick	T1–T5 Med tick
f1 Sphere	no	0.0%	0.0%	0.0%	—	—
f2 Ellipsoid	no	13.3%	40.0%	**+26.7%**	169	138
f15 Rastrigin	yes	0.0%	0.0%	0.0%	—	—
f24 Lunacek	yes	0.0%	0.0%	0.0%	—	—

Final normalized fitness at tick 200 — Median [Q1, Q3]:

Function	T1–T4 NF	T1–T5 NF	T5 Advantage
f1 Sphere	0.134 [0.082, 0.187]	0.136 [0.067, 0.188]	1.0×
f2 Ellipsoid	0.210 [0.067, 0.392]	0.021 [0.005, 0.038]	**10.0×**
f15 Rastrigin	0.405 [0.351, 0.481]	0.406 [0.368, 0.481]	1.0×
f24 Lunacek	0.533 [0.449, 0.602]	0.287 [0.245, 0.360]	**1.9×**

T5 rewire selectivity: T5 fired 0 proposals on f1 (stagnation never triggered — greedy descent makes continuous progress on the symmetric sphere), accepted 39/838 on f2 (4.7%), 23/2519 on f15 (0.9%), and 49/1949 on f24 (2.5%). The low accept rate is a design feature: T5 discards rotations that do not improve fitness, producing an outcome-gated lineage.

Interpretation:

f1 Sphere: Rotationally invariant — any basis rotation yields an equivalent landscape. T5 correctly abstains (0 proposals fired). The 0% convergence for both conditions reflects a budget limitation: 200 ticks of coordinate-wise greedy descent in 10D does not reliably reach NF ≤ 0.01 from a uniform-random initialization. Both conditions converge comparably.

f2 Ellipsoid (ill-conditioned): The 10^6 condition number creates a narrow ellipsoidal valley that coordinate-wise descent (T1) cannot follow efficiently. A T5 basis rotation aligns the search axes with the valley's principal direction, enabling the T1–T4 stack to exploit it. The result is a **10× median NF reduction** and **+26.7 pp convergence rate** improvement. The lineage (`experiments/bbob/evidence/lineage.md`) records each accepted rewire with

tick, generation, fitness-before, and fitness-after — a compounding pattern visible across accepted events (e.g., Trial 1: rewire at tick 67, NF 0.056 → 0.012; subsequent T1–T4 descent reaches NF 0.002).

f15 Rastrigin (densely multimodal): At DIM=10, Rastrigin has ~10^10 local optima on a regular grid. Each rotation lands in a new basin of comparable depth; improvements rarely compound within 200 ticks. The 0.9% accept rate reflects this — rewires are occasionally accepted when they chance upon a marginally better basin, but the gain is not systemic. This is an **honest null result** within the given budget: T5 is not universally superior, and the experiment correctly surfaces the boundary.

f24 Lunacek (bimodal): T1–T4 reliably converge to the secondary (sub-optimal) basin. T5 rewires occasionally reorient the search toward the global basin, reducing median final NF by **46%** (0.533 → 0.287) without achieving full convergence within 200 ticks. A longer budget (≥500 ticks) is predicted to produce convergence rate separation comparable to f2.

What this establishes: The T5 structural primitive is (a) selective — it rejects rotations that do not improve fitness, (b) *load-bearing* on ill-conditioned landscapes where parameter adjustment alone cannot resolve the conditioning artifact, and (c) directionally beneficial on bimodal landscapes where inter-basin structure requires a structural jump. It is not a universal improvement — the f1 and f15 results confirm falsifiability. This matches the theoretical claim in §2: T5 escape is load-bearing when the fitness landscape contains inter-basin topology that parameter adjustment cannot traverse.

4.6 AEGIS Controlled Experiment — Inter-Generational Meiosis

The BBOB experiment in §4.5 validates the T5 structural primitive (basis-rotation `StructuralRewire`) on a continuous optimisation benchmark, but it does not demonstrate *inter-generational meiosis* — the case where the accepted topology becomes the genome for the next epoch and T5 fires again at the epoch boundary. This section closes that gap with the AEGIS delta-neutral DeFi strategy, a domain where T1–T4 provably cannot cross the inter-basin fitness valley and T5 operates through state-machine topology switching at epoch boundaries.

Setup. The AEGIS being manages a delta-neutral position: AAVE V3 collateral loan (ETH→USDC), Uniswap V3 concentrated LP (±5% ETH/USDC), and a Hyperliquid perpetual short. The strategy has two attractors: *LP-Active* (lower MTS basin, MTS≈0.35, hedge ratio 0.80, LP capital 65%) and *LP-Bypassed* (upper MTS basin, MTS≈0.65, no LP, hedge ratio 0.0). E88 canonical params: Sharpe=1.02, Return=+213.6%, MaxDD=33.4%. **Origin of E88 parameters.** The E88 canonical parameter set was identified through approximately 100 manual parameter explorations by the author over several weeks; §4.6 validates the meiosis *mechanism* operating on that configuration, not the autonomous discovery of the configuration itself. This is intentional: the controlled experiment isolates the structural primitive (inter-generational topology switching) from the surrounding question of

whether BIOISO can autonomously locate optimal parameters in a search space the author has already exhausted manually. The latter is a separate, larger experiment.

The fitness landscape is bimodal in the `(hedge_ratio, lp_capital_pct)` plane. The inter-basin valley corresponds to parameter regions where LP is partially active (capital 5–40%) — neither earning meaningful fees nor providing delta-neutrality — producing suboptimal performance in every regime. T1 gradient perturbation and T2-class CMA-ES (consistent with the §6.4 classification) both operate within topology-bounded feasible sets: LP-Active bounds `hedge_ratio` ∈ [0.60, 1.00] and `lp_capital_pct` ∈ [0.40, 0.90]. These bounds prevent T1–T4 from constructing the parameter trajectory required to exit to LP-Bypassed.

Experiment. 10 trials × 5 epochs × 200 ticks/epoch. Epoch schedule: Ranging → MildBull → StrongBull → Ranging → MildBear. T5 probes at each epoch boundary using an analytical Sharpe comparison with calibrated estimation noise (σ=0.25), accepting if the alternative topology exceeds the current by >0.10 Sharpe. Analytical acceptance rates: Ranging (<2%), StrongBull (~91%), MildBear (<2%). Seeds are `wrapping_mul(i, 0x517CC1B727220A95)+1` for trial i (reproducible; no cherry-picking).

Scope of the validation — what this experiment is and is not. The AEGIS experiment validates the meiosis *mechanism* under an analytical Sharpe model calibrated to LP-fee, impermanent-loss, carry, and drift economics; it does **not** validate the strategy under live market execution. The analytical model establishes the *expected* acceptance rate per regime (Ranging <2%, StrongBull ~91%, MildBear <2%) and the experiment confirms the realized 10-trial rates land within the analytical band under the declared σ=0.25 noise. Because the analytical model both predicts the rates and generates the per-tick fitness signal the meiosis gate operates on, this is a closed-system validation of the mechanism (does the gate fire when the model says it should? does the topology transition correctly when the model says regimes have shifted?) rather than an open-system validation of the strategy's performance against unmodeled market dynamics (slippage, gas spikes, oracle latency, MEV, protocol risk, depegging events, regulatory action). A reviewer interpreting these results should read them as: "the meiosis mechanism functions as designed under the analytical model" — not "the AEGIS strategy with T5 outperforms by +0.517 Sharpe in production." Live-trading validation is forthcoming work and requires a separate experimental setup (paper trading on a testnet over a multi-month horizon with real on-chain price feeds and gas costs).

Epoch	Regime	T1–T4 Sharpe	T1–T5 Sharpe	Δ Sharpe	Rewires/10
0	Ranging	2.162	2.006	−0.156	0/10
1	MildBull	2.565	2.777	+0.213	0/10
2	StrongBull	2.002	2.519	**+0.517**	**10/10**
3	Ranging	2.488	2.149	−0.339	10/10
4	MildBear	−0.572	−0.858	−0.285	1/10

Cumulative 5-epoch mean Sharpe: T1–T4 = 1.710, T1–T5 = 1.686 (**Δ = −0.024** — net loss).

StrongBull (epoch 2): ETH appreciating at +350%/yr annualised causes the ±5% LP range to exit range within ~6 ticks, spending 80% of the epoch out-of-range. LP earns fees on 20% of capital and incurs 20%/yr impermanent loss. LP-Active expected Sharpe = 1.90; LP-Bypassed expected Sharpe = 2.48. Gap = 0.58 Sharpe. Observed: 10/10 trials accepted the topology switch; median per-epoch Sharpe advantage = **+0.517**. T1–T4 cannot replicate this: the fitness valley between topologies lies outside the LP-Active feasible set.

Regime detection is calibrated: 0/10 false positives in Ranging (epoch 0); 10/10 correct detections in StrongBull (epoch 2); 10/10 correct back-switches in the following Ranging epoch (epoch 3). The 1/10 false positive in MildBear (trial 6) is consistent with calibration: both topologies have negative expected Sharpe (LP-Active ≈ −0.27, LP-Bypassed ≈ −2.53), and the marginal probe reading (+0.14, just above the 0.10 threshold) is plausible under σ=0.25 estimation noise.

T5 is worse than T1–T4 in epoch 4 (MildBear). The table is honest about this: T1–T5 Sharpe is −0.858 vs T1–T4's −0.572 — T5 loses an additional 0.286. The mechanism is that in regimes where both topologies have negative expected Sharpe, a T5 false-positive topology switch trades a small drawdown (LP-Active −0.27) for a larger one (LP-Bypassed −2.53) plus a re-convergence cost. This is a real cost the framework does not paper over: T5 is a *upside-regime detection* mechanism, not a bear-regime optimization mechanism. The acceptable use of T5 is where the upside-regime gain (StrongBull: +0.517) statistically dominates the bear-regime cost (MildBear: −0.286) over the deployment horizon. In a regime mix where bear epochs outnumber StrongBull epochs by a wide enough margin, T5 should be disabled — the framework supplies the detection mechanism; the deployment policy (when to enable T5 versus pinning to T1–T4) is a domain decision, not a framework default.

Inter-generational compounding and transition cost. Each epoch boundary is a genome cycle: the accepted topology is carried forward. In epoch 3 (Ranging), T1–T5 correctly switches back to LP-Active (10/10) but pays a **parameter re-convergence cost** (−0.339 Sharpe): after the topology switch, parameters start near the LP-Bypassed optimum and require 100–150 ticks to re-converge to the LP-Active optimum. T1–T4, never having left LP-Active, arrives with fully-converged parameters. This cost is real and is not suppressed. **Over the 5-epoch experiment the net is −0.024 Sharpe — a small loss, not a gain.** The loss decomposes as: epoch-2 StrongBull win (+0.517) offset by epoch-3 return-Ranging re-convergence cost (−0.339) yielding +0.178 per (StrongBull + return-Ranging) cycle; plus epoch-4 MildBear false-positive (−0.285); plus epoch-0/1 small per-epoch deltas (−0.156 / +0.213) that roughly cancel — summing to the observed net loss.

Compounding projection (with explicit sign convention). Per (StrongBull + return-Ranging) cycle the additive contribution to cumulative Sharpe is:

> per-cycle Δ = (StrongBull win) + (return-Ranging cost) = (+0.517) + (−0.339) = **+0.178 Sharpe**

Over k such cycles with *no* bear regime, the projected net is $k \times 0.178$ — monotonically increasing. The 5-epoch experiment contains exactly **one** such cycle (epoch 2 → epoch 3), plus a MildBear epoch the cycle decomposition does not account for. A multi-year backtest with multiple StrongBull/Ranging cycles is required to demonstrate the projected positive cumulative — and even then, the projection holds only when bear-regime false-positive cost (epoch-4 in this experiment) does not statistically dominate the per-cycle gain. See the bear-regime acceptability note in §4.6 below.

Lineage. The full rewire lineage is recorded in `experiments/aegis/evidence/lineage.md`. Each accepted rewire is tagged with trial, generation, regime, topology transition, and probe Sharpe before/after. The per-trial Δ ranges from −0.580 (trial 7, unlucky noise in epoch 3) to +0.379 (trial 4), reflecting the σ=0.25 per-epoch estimation noise. Across 10 trials, 21 total accepted rewires: 10 in StrongBull, 10 in the return-Ranging, 1 false-positive in MildBear.

What this establishes. AEGIS closes the inter-generational meiosis gap: (a) T5 fires at epoch boundaries, not within ticks; (b) the accepted topology is transmitted as the genome for the next generation; (c) the mechanism produces measurable Sharpe advantage in a regime T1–T4 cannot access; (d) detection rate matches the analytical model; (e) transition costs are real and quantified. The experiment does not require a live trading environment — the analytical Sharpe model is grounded in LP fee, impermanent loss, carry, and drift economics, all calibrated to the E88 parameter set.

---

5. Ten Seeded Domains: One T5 Validated, Seven Theoretically Motivated, Two Calibration

The BIOISO framework seeds ten domains. The split, made explicit:

- **§5.8 `aegis_delta_neutral`** — the one T5 domain empirically validated in this paper (§4.6).
- **§5.1–§5.7** — seven T5 domains theoretically motivated below. Each satisfies the structural criterion for T5 (the optimal solution class changes during the experiment horizon, making `StructuralRewire` rather than `ParameterAdjust` the load-bearing primitive). Empirical validation analogous to §4.6 is forthcoming work for each.
- **§5.9 `biosphere`** (T4 calibration) and **§5.10 `ocean_circulation`** (T3 calibration) — deliberately *not* BIOISO/T5 entities. They are included to demonstrate that the framework correctly assigns lower tiers where they are sufficient, providing the falsifiability boundary for the framework's "T5 is necessary" claim.

A separate controlled experiment on the COCO/BBOB f2 ill-conditioned ellipsoid (§4.5) validates the T5 *primitive* on a standard public benchmark; it is not itself a "BIOISO domain" in the sense of §5 but provides reproducible evidence on a non-domain landscape.

For each domain we present (a) why T1–T4 are predicted to saturate, (b) the T5 mechanism that addresses the saturation, and where applicable (c) the academic baseline that establishes the structural-criterion bar. Empirical-status is summarized in the domain table at the end of §5.

BIOISO domains satisfy three criteria: (1) the fitness landscape is coevolutionary or structurally non-stationary; (2) `StructuralRewire` is load-bearing — `ParameterAdjust` cannot converge; (3) the problem is currently unsolved or inadequately addressed at T1–T4.

5.1 `amr_coevolution` — Antimicrobial Resistance

Why T1–T4 saturate: AMR pathogens evolve resistance mechanisms on a timescale of hours to days. A T4 GP surrogate trained on resistance mechanisms from generation `n` will have misspecified priors for generation `n+1` because the target protein has structurally mutated. The GP cannot represent a binding hypothesis that did not exist in its training data.

T5 mechanism: `StructuralRewire` replaces the pharmacophore hypothesis class when the surrogate's predictive variance exceeds threshold — the being generates a new hypothesis topology rather than refining parameters within the old one. Meiosis bakes the new hypothesis template into the next genome.

Reference baseline: AlphaFold 2 (Jumper et al. 2021) provides structure prediction; BIOISO provides the adaptive strategy for selecting which structures to target as resistance evolves.

5.2 `flash_crash` — HFT Market Microstructure

Why T1–T4 saturate: HFT firms reverse-engineer and game fixed circuit breaker rules within hours of deployment (Kirilenko et al. 2017). A T3 hyper-heuristic with a fixed portfolio of detection rules will have all portfolio members neutralized by adversarial trading within a single trading session. T4's GP cannot generate detection logic for attack patterns it has never observed.

T5 mechanism: The `flash_crash` BIOISO entity generates novel detection signal categories — not parameter tuning of existing signals, but structural synthesis of new signal types that the adversarial strategy has not yet been designed to evade. The meiosis loop promotes detection categories that survived a full trading session without being gamed.

5.3 `adaptive_jit` — JIT Compiler Optimization

Why T1–T4 saturate: The optimal IR pass sequence for a JIT compilation target changes as the runtime hot-path profile evolves. A T4 surrogate trained on pass orderings for workload `W_n` has an architectural mismatch for workload `W_{n+1}` if the hot-path structure changes.

T5 mechanism: `StructuralRewire` replaces which compiler transformation passes compose and in what order — not parameter tuning within a fixed pipeline, but synthesis of a new pipeline topology that the hot-path profile calls

for.

5.4 `protein_drug_resistance` — Cancer/HIV Drug Resistance

Why T1–T4 saturate: Cancer and HIV drug targets mutate under selection pressure from existing drugs (Perelson et al. 1997). Each drug-target binding hypothesis is architecturally specific to the target's current structure. As the target mutates, the hypothesis becomes incorrect — not merely suboptimal, but structurally wrong about which sites are binding candidates.

T5 mechanism: The BIOISO generates new binding hypotheses from structural analysis of the mutated target, replacing the hypothesis topology rather than refining its parameters.

5.5 `ics_zero_day` — ICS Zero-Day Defense

Why T1–T4 saturate: Zero-day attacks have, by definition, no prior instances in training data. A T4 attention model trained on known attack patterns has zero generalization to attacks with novel structure — the architecture encodes a prior over the known attack class distribution that is wrong for zero-days.

T5 mechanism: The BIOISO synthesizes new signal detection categories by hypothesis generation — it proposes signal correlations that would indicate a structurally novel attack, validates them against the current signal stream, and promotes detection logic that fires on the novel pattern.

5.6 `quantum_error_mitigation` — Quantum Circuit Compilation

Why T1–T4 saturate: Quantum hardware noise models change per calibration cycle, making the optimal gate decomposition strategy non-stationary (Preskill 2018). A T4 surrogate fitted to the noise model at calibration time `t` is misspecified after recalibration at `t+1` if the noise structure changes.

T5 mechanism: `StructuralRewire` discovers new circuit decomposition strategies by structural mutation of the gate sequence template — not parameter optimization within a fixed template, but replacement of the template topology when the noise structure changes.

5.7 `climate_intervention` — Earth System Intervention Sequencing

Why T1–T4 saturate: Each deployed climate intervention changes the causal structure of the Earth system (Lenton et al. 2019). Solar radiation management at time `t` alters cloud feedback mechanisms; subsequent interventions operate on a causally different system. A T4 GP fitted to the pre-intervention causal graph has systematically wrong priors post-intervention.

T5 mechanism: The BIOISO structurally adapts which interventions to sequence by replacing the causal model template when prior interventions alter system coupling beyond the GP's predictive capacity.

5.8 `aegis_delta_neutral` — DeFi Delta-Neutral Strategy Evolution

Why T1–T4 saturate: DeFi liquidity pool dynamics are coevolutionary — each deployed strategy changes the on-chain liquidity distribution, making the fitness landscape of subsequent strategies a function of prior ones (analogous to arms-race coevolution in ecology). A T4 GP trained on historical pool states has systematically wrong priors when the liquidity topology changes due to protocol upgrades, new pool deployments, or adversarial MEV. The canonical E88 parameters (mean taper step = 0.35, +213.6% return, Sharpe = 1.02) were optimal for a specific liquidity regime; they saturate when the regime shifts.

T5 mechanism: The `aegis_delta_neutral` BIOISO entity uses `StructuralRewire` to replace which hedging instrument class is used for delta neutralization — not tuning the hedge ratio within a fixed class, but synthesizing a new hedging strategy topology when the GP's predictive variance exceeds a threshold. Meiosis bakes the surviving hedging architecture into the next genome, compounding structural improvements across market regimes.

Reference baseline: AEGIS E88 canonical parameters: `aave_target_health_factor = 1.85`, `uniswap_lp_range_pct = 12.0%`, mean taper step = 0.35. These represent the T4-generation optimum before structural adaptation. BIOISO T5 escape replaces the strategy architecture when the Sharpe ratio degrades below 0.7 for `k` consecutive rebalancing cycles.

5.9 `biosphere` — Biodiversity Metric Optimization (T4, Calibration Domain)

A T4 calibration domain included to establish where T5 is *not* load-bearing. The GP surrogate models the response of biodiversity indices to conservation interventions. T4 is the appropriate ceiling because the biodiversity landscape, while non-stationary, has stable causal structure across the relevant policy horizon (decades). The optimal intervention *class* does not change structurally within the policy window — only its parameters do.

Including this domain establishes the falsifiability boundary: the BIOISO model claims T5 is necessary only for domains where structural rewiring is load-bearing. A framework that ran T5 everywhere would prove nothing.

5.10 `ocean_circulation` — Ocean Circulation Homeostasis (T3, Calibration Domain)

A T3 calibration domain establishing the lower bound of the taxonomy. The hyper-heuristic selects between thermohaline intervention operators based on observed circulation indices. The operator portfolio (temperature regulation, salinity adjustment, current redirection) covers the causal mechanism space adequately; the T3 ceiling does not apply because the AMOC's structural dynamics do not exit the portfolio's coverage within the intervention horizon.

These two calibration domains are not BIOISO (T5) entities — they are included to demonstrate tier placement correctness: the framework assigns T3 or T4 where those tiers are sufficient, not T5 by default.

Domain summary:

Domain	Tier	T5 reason	Structural criterion (Y/N)[2]	Empiri status (pape
`amr_coevolution`	T5	Pathogen evolves binding targets structurally	Y (target structure mutates)	Theoreti motivate
`flash_crash`	T5	Adversarial gaming invalidates all fixed detection rules	Y (adversary defeats fixed portfolio)	Theoreti motivate
`adaptive_jit`	T5	Hot-path profile changes IR pass topology	Y (workload topology shifts)	Theoreti motivate
`protein_drug_resistance`	T5	Target mutation makes hypothesis class wrong	Y (binding hypothesis class becomes wrong)	Theoreti motivate
`ics_zero_day`	T5	Zero-days have no training-data ancestors	Y (no prior class of training instance)	Theoreti motivate
`quantum_error_mitigation`	T5	Recalibration changes gate decomposition topology	Y (noise model changes between calibrations)	Theoreti motivate
`climate_intervention`	T5	Intervention changes causal graph structure	Y (each intervention re-wires the causal model)	Theoreti motivate
`aegis_delta_neutral`	T5	Liquidity topology coevolves with strategies	Y (LP-Active ↔ LP-Bypassed inter-basin valley)	**Empiric validate (§4.6)**

Domain	Tier	T5 reason	Structural criterion (Y/N)[2]	Empiri status (pape
`biosphere`	T4	Stable causal structure; GP-UCB sufficient	N (causal structure stable in horizon)	Calibrati domain (placeme
`ocean_circulation`	T3	Fixed operator portfolio covers mechanism space	N (mechanism space covered by portfolio)	Calibrati domain (placeme

A separate controlled experiment on the COCO/BBOB f2 ill-conditioned ellipsoid (§4.5) validates the T5 structural primitive on a standard public benchmark; it is not itself a "BIOISO domain" in the sense above but provides reproducible evidence on a non-domain landscape.

The eight motivated T5 domains are not selected because they are prominent — they are selected because they satisfy the structural criterion: the optimal solution's *class* changes during the experiment horizon, making T1–T4 saturation a mathematical consequence rather than a practical limitation. **For each domain, an empirical validation analogous to §4.6 (AEGIS) is forthcoming work**; without that validation, the present paper claims theoretical motivation only — the structural criterion is satisfied, and the T5 mechanism is specified, but performance on the live domain is not measured here.

Implementation-vs-spec note on the §5 roster. The current `src/runtime/bioiso_runner.rs` registers eight runtime BIOISO entities as T5: seven match §5.1–§5.7 by name (`amr_coevolution`, `flash_crash`, `adaptive_jit`, `protein_drug_resistance`, `ics_zero_day`, `quantum_error_mitigation`, `climate_intervention`), plus `aegis_delta_neutral` from §5.8 (the empirically validated one), plus two additional T5 entities introduced during development that are **not** in §5: `fusion_plasma` (plasma-confinement control with non-stationary noise topology) and `adaptive_self_assembly` (nanostructure protocol-graph rewiring). The two calibration domains described in §5.9 (biosphere T4) and §5.10 (ocean_circulation T3) are described in this paper as the framework intends them but are **not yet implemented as runtime entities** in `bioiso_runner.rs`; their inclusion here is to establish the framework's falsifiability boundary — implementation of the calibration domains as runtime entities is forthcoming work, listed in §8 Limitations. The runtime/paper roster mismatch is acknowledged here in the open rather than papered over.

---

6. Related Work

6.1 Hyper-Heuristics

Burke et al. (2013) survey the state of hyper-heuristics as of 2013. The field distinguishes *selection* hyper-heuristics (our T3) from *generation* hyper-heuristics (closer to T5 in spirit, but without cross-generational persistence). The key distinction between T5 and generation hyper-heuristics: generation HHs generate new low-level heuristics within a single run; BIOISO promotes structural mutations across compiled generations via meiosis. The genome is the persistence mechanism.

6.2 Bayesian Optimization

Srinivas et al. (2010) prove sublinear regret bounds for GP-UCB. Our T4 ceiling theorem does not contradict these bounds — it identifies the condition under which the bounds' assumptions fail: when the objective is outside the kernel's RKHS. Snoek et al. (2012) make BO practical for hyperparameter tuning; the ceiling we identify is their practical observation that "sometimes you need to rethink the search space," formalized.

6.3 Neural Combinatorial Optimization

Kool et al. (2019) train an attention model to solve routing problems with near-optimal performance. The T4 ceiling for attention models is that the trained policy is fitted to a specific problem distribution; distribution shift that changes the combinatorial structure of the optimal solution requires architectural change, not weight update.

6.4 Evolutionary Strategies and AutoML

CMA-ES (Hansen & Ostermeier 2001) is a sophisticated T2-class algorithm. AutoML (Hutter et al. 2019) is a T4-class framework (GP/SMAC for algorithm configuration search). Neither crosses the T5 boundary: they optimize algorithm parameters, not algorithm class. The defining characteristic of T5 is that the algorithm class — the hypothesis space itself — is the subject of optimization.

6.5 Meta-Learning and Algorithm Selection

Vanschoren (2018) surveys meta-learning. The algorithm selection problem (Rice 1976) asks: given a portfolio of algorithms, which one to apply to a given instance? This is T3 — selection from a fixed portfolio. T5 generates new portfolio entries.

6.6 Program Synthesis

AlphaDev (Mankowitz et al. 2023) discovers new sort algorithms via RL on assembly code. This is structurally close to T5 but operates in a different regime: AlphaDev discovers algorithms within a fixed computational substrate (assembly), without the cross-generational persistence mechanism (meiosis) or the biological lifecycle constraint (telomere). BIOISO provides the lifecycle and persistence model.

---

7. Discussion

7.1 The Meiosis Requirement

The defining T5 primitive is not `rewire:` alone — it is the combination of `rewire:` (intra-generational structural replacement) with meiosis (inter-generational mutation promotion). Without meiosis, `rewire:` produces only transient structural change: the next entity spawned from the same genome would start from the pre-rewire baseline. With meiosis, structural mutations compound: each generation begins from the endpoint of the previous generation's learning, not its start.

This is the formal analog of the distinction between Lamarckian inheritance (acquired characteristics pass to offspring) and Darwinian selection (only genotype-encoded variants pass). BIOISO meiosis is Lamarckian at the genome level — the promoted mutations are literally written into the next genome specification — but Darwinian at the fitness level: only mutations that reduced drift in the current generation are promoted.

7.2 Safety Constraints on Structural Change

T5 entities require `@mortal @corrigible @sandboxed @auditable` annotations (the SafetyChecker enforces these as compile errors). This is not decoration — it is the safety invariant for structural self-modification:

- `@mortal` + `telomere:` bounds the scope of intra-generational change.
- `@corrigible` requires `modifiable_by: human_operator` in `telos:` — a human can override the telos at any point.
- `@sandboxed` restricts the entity's signal surface: a structurally self-modifying entity must have a declared boundary.
- `@auditable` requires a telomere audit log: every structural mutation, every meiosis event, every genome promotion is recorded and traceable.

The auditable constraint makes the T5 escape *inspectable*: the lineage graph from genome `G_0` to `G_n` is a complete record of which structural mutations accumulated and why each was promoted.

7.3 The Language Level Matters

The BIOISO model requires a language-level implementation for a specific reason: the genome is a source specification, not a runtime data structure. Meiosis modifies the genome and recompiles — this is a compiler operation, not a runtime operation. A runtime-only system (a Python dict of algorithm parameters) cannot implement true T5 meiosis because it cannot change which algorithm the system compiles to in the next generation.

Loom's role is to make the genome a typed, verifiable specification from which all runtime behavior is derived. The GS methodology's derivability constraint ensures that a stateless reader (the meiosis engine) can apply genome mutations correctly without prior context. This is the connection between the language design (GS-derivable specifications) and the evolutionary mechanism (GS-derived genome application).

---

8. Limitations

Benchmark scale. The `bench_colony_ladder` empirical demonstration runs 60 ticks on a single-machine scheduling problem with a synthetic non-stationary drift signal `D(t, ϕ) = 0.3|sin(0.05t + ϕ)| + 0.05ε`. This is sufficient to demonstrate T1→T4 auto-escalation but not inter-generational meiosis. Real BIOISO domains (AMR coevolution, climate intervention) operate on horizons of months to years; the 60-tick benchmark is a proof-of-mechanism, not an evaluation of domain performance. The BBOB experiment in §4.5 (30 trials × 200 ticks × 4 functions) validates the T5 structural primitive on a standard public benchmark and shows clear advantage on ill-conditioned (f2: 10× NF) and bimodal (f24: 1.9×) landscapes. The AEGIS experiment in §4.6 (10 trials × 5 epochs × 200 ticks) demonstrates inter-generational meiosis on the AEGIS DeFi domain, with topology switches firing at epoch boundaries and the accepted topology transmitted as genome for the next generation. The f15 and f24 full-convergence results are budget-limited (200 ticks is insufficient for DIM=10 Rastrigin/Lunacek). The AEGIS 5-epoch cumulative result is a small **loss** (−0.024 Sharpe), not a gain — the StrongBull win (+0.517) is more than offset by the return-Ranging re-convergence cost (−0.339) and the MildBear false-positive (−0.285). The framework's claim is not that T5 wins on every regime mix, but that per (StrongBull + return-Ranging) cycle the additive contribution is +0.178 Sharpe; a multi-year backtest with multiple such cycles and a bear-light regime mix is required to demonstrate compounding advantage at scale.

Meiosis requires recompilation infrastructure. The T5 meiosis loop operates via a GitHub Actions workflow (`.github/workflows/evolve.yml`) and requires `cargo test` to pass after each genome application. This makes T5 practically limited to environments where recompilation is feasible — it is not a runtime mechanism. Systems without a build pipeline cannot operate at T5.

Meiosis gate is composed across three locations, not one. §3.3 specifies a three-filter gate (improvement + safety + stability). In the current implementation only the stability filter (cargo build + `cargo test --lib`) is directly enforced by `.github/workflows/evolve.yml`. The improvement filter

is enforced upstream in the runtime `MeiosisEngine`; the safety filter is enforced indirectly by the loom compiler's `SafetyChecker`. A reviewer inspecting only `evolve.yml` will see only the stability stage. The pipeline is the *composition* of all three locations; consolidating them into a single explicit gate stage at the workflow boundary is forthcoming infrastructure work and is a known scoping gap between specification and implementation.

Runtime T4 is a bandit UCB, not GP-UCB. §2.5 defines T4 as Gaussian-process or attention-model surrogate optimization. The runtime function dispatched as T4 in `solver_tiers.rs` is a bandit-style UCB (`μ + βV(ln(N+1)/(n+1))`) without a Gaussian-process posterior or any kernel-based surrogate. The §2.5 ceiling argument applies to the *conceptual* T4 (GP-UCB); the runtime T4 has a different, weaker ceiling. A full GP-UCB implementation is forthcoming work. See §4.2 implementation note for details.

Domain implementation roster mismatch. The current `src/runtime/bioiso_runner.rs` registers eight runtime BIOISO entities as T5 (the seven §5.1–§5.7 motivated domains plus §5.8 AEGIS), but also includes two T5 entities not described in §5 (`fusion_plasma`, `adaptive_self_assembly`) and omits the two §5.9–§5.10 calibration domains (`biosphere` T4, `ocean_circulation` T3). The mismatch is acknowledged in §5; aligning the runtime roster with the paper's roster (either by adding the calibration domains to the runtime or by replacing §5.9–§5.10 with `fusion_plasma`/`adaptive_self_assembly`) is forthcoming work.

Domain-saturation arguments are generic, not domain-specific (§5.1–§5.7). For each of the seven theoretically-motivated T5 domains in §5, the argument that T1–T4 saturate is currently stated in the general form "the optimal solution class changes during the experiment horizon, so the T4 prior misspecifies after the shift." A reviewer evaluating any individual domain (e.g., adaptive_jit, ics_zero_day) will rightly ask for the *domain-specific* structural-impossibility argument — why, concretely, the JIT pass orderings cannot be enumerated in a fixed portfolio for a given hardware target, or why zero-day attack signatures cannot be parameterized within a T4 surrogate's kernel. Producing these domain-specific arguments (each requires domain expertise the present paper's author may not bring alone) is forthcoming work; the current §5 entries should be read as initial framings, not final arguments.

Property 2 (structural escape) assumes adequate candidate pool. The existence proof in §3.4 holds given that the `rewire: candidates:` pool includes an algorithm class that can converge on the target problem. If the initial candidate pool is architecturally wrong for the problem — e.g., all candidates are parameter-adjustment strategies for a problem requiring structural graph replacement — no meiosis cycle will converge. Pool design is a human responsibility.

Ceiling theorems are structural, not statistical. The T1–T4 ceiling theorems establish that a class of structural modifications cannot be expressed within each tier — they do not make probabilistic claims about convergence rates on specific instances. Whether a T5 entity out-performs a T4 entity on a given problem in practice depends on the specific instance, the candidate pool, and the generation budget.

Domain baselines are simulated. The CEMS runtime runs signal simulators calibrated to academic baselines, not live data streams. Results do not constitute deployment-validated evidence for any specific application domain.

Empirical evidence covers one T5 domain plus one controlled primitive experiment. Of the eight T5 domains in §5 (§5.1–§5.8), only `aegis_delta_neutral` (§5.8) is empirically validated in this paper, via §4.6. The COCO/BBOB f2 experiment (§4.5) validates the T5 *primitive* on a standard public benchmark but is not itself a BIOISO domain. The remaining seven T5 domains (§5.1–§5.7) are theoretically motivated — the structural criterion is satisfied and the T5 mechanism is specified, but their empirical validation is forthcoming work. The two calibration domains (§5.9–§5.10) are tier-placement examples, not T5 entities. A reader skeptical of the framework should weight the empirical claims accordingly.

Formal biological isomorphism is not demonstrated in this paper. The "biologically-inspired" framing in §3.5 acknowledges structural inspiration, not formal organizational-closure equivalence with biological autopoiesis (Maturana & Varela 1972). Establishing the formal isomorphism — that BIOISO's `(G, T, M, Ω)` structure satisfies the autopoietic criteria as formalized by Maturana–Varela and subsequent work (Di Paolo 2005; Bianchini 2023) — is the subject of a companion paper currently in preparation; that paper will require biological-systems expertise this paper's author does not bring alone. The broader autopoietic argument (BIOISO and Loom as instances of *directed formal autopoiesis*) is developed in the companion essay *Onwards* (Ghiringhelli 2026); see §3.5 for the relationship between the three works.

---

9. Conclusion

The BIOISO framework proposes a five-tier hierarchy of optimization entities, with T5 being the first tier whose adaptations operate on the space of algorithms rather than the space of parameter values, operator selections, or surrogate model weights. The T5 escape mechanism — structural self-modification via meiosis — is the first mechanism that compounds algorithmic improvements across generations without requiring a human developer to write new source code each cycle.

The implementation in Loom demonstrates that this framework can be expressed as a typed, verifiable specification: `learn:`, `plasticity:`, `rewire:`, and `telomere:` are first-class keywords with checker rules, parser implementations, and runtime dispatch logic. The `examples/ladder.loom` file provides a compilable specification of the T1→T5 progression for job scheduling. The `bench_colony_ladder` binary demonstrates intra-generational auto-escalation; the BBOB experiment (§4.5) validates the T5 structural primitive on a public benchmark with reproducible advantage on ill-conditioned landscapes; the AEGIS experiment (§4.6) closes the inter-generational meiosis gap with measured Sharpe advantage in the regime where T1–T4 cannot operate.

Status. This is preprint v1. The eight theoretically-motivated domains in §5 await empirical validation analogous to §4.6. The formal autopoietic isomorphism — establishing organizational-closure equivalence between BIOISO and the Maturana–Varela definition — is the subject of a companion paper in preparation. The broader theoretical framing (the formal tradition waiting for its executor; the AI as Logos to the formal Nous; directed formal autopoiesis as the structural category) is developed in the companion essay *Onwards* (Ghiringhelli 2026, submitted to ACM SIGPLAN Onward! 2026). Readers seeking the theoretical scaffolding should read the three works together; readers seeking proof of formal isomorphism should wait for the forthcoming companion paper.

References

- Brélaz, D. (1979). New methods to color the vertices of a graph. *Communications of the ACM*, 22(4), 251–256.
- Burke, E.K., et al. (2013). Hyper-heuristics: A survey of the state of the art. *Journal of the Operational Research Society*, 64(12), 1695–1724.
- Ghiringhelli, J.C. (2026). *Loom: An AI-Native Functional Language* — repository and canonical manual. https://github.com/jghiringhelli/loom (see `docs/manual.md` for the language reference).
- Ghiringhelli, J.C. (2026). Generative Specification: A Pragmatic Programming Paradigm for the Stateless Reader. *Pragmaworks Preprint.* https://doi.org/10.5281/zenodo.19637142
- Ghiringhelli, J.C. (2026). Onwards: The Formal Tradition Was Waiting for Its Executor. *Submitted to ACM SIGPLAN Onward! 2026.*
- Maturana, H.R., & Varela, F.J. (1972). *De Máquinas y Seres Vivos: Autopoiesis — La Organización de lo Vivo.* Editorial Universitaria.
- Di Paolo, E.A. (2005). Autopoiesis, Adaptivity, Teleology, Agency. *Phenomenology and the Cognitive Sciences*, 4(4), 429–452.
- Bianchini, F. (2023). Autopoiesis of the Artificial: From Systems to Cognition. *BioSystems*, 234, 105065.
- Hansen, N., & Ostermeier, A. (2001). Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2), 159–195.
- Hansen, N., Finck, S., Ros, R., & Auger, A. (2009). Real-parameter black-box optimization benchmarking 2009: Noiseless functions definitions. *INRIA Research Report RR-6829.*
- Holland, J.H. (1975). Adaptation in Natural and Artificial Systems. University of Michigan Press.
- Hutter, F., Kotthoff, L., & Vanschoren, J. (2019). Automated Machine Learning. Springer.
- Johnson, D.S. (1974). Fast algorithms for bin packing. *Journal of Computer and System Sciences*, 8(3), 272–314.
- Jumper, J., et al. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596, 583–589.
- Kirilenko, A.A., et al. (2017). The flash crash: High-frequency trading in an electronic market. *Journal of Finance*, 72(3), 967–998.
- Kirkpatrick, S., Gelatt, C.D., & Vecchi, M.P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680.

- Kool, W., van Hoof, H., & Welling, M. (2019). Attention, learn to solve routing problems! *ICLR 2019.*
- Lenton, T.M., et al. (2019). Climate tipping points — too risky to bet against. *Nature*, 575, 592–595.
- Lourenço, H.R., Martin, O.C., & Stützle, T. (2003). Iterated local search. In *Handbook of Metaheuristics* (pp. 320–353). Kluwer.
- Mankowitz, D.J., et al. (2023). Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618, 257–263.
- Perelson, A.S., et al. (1997). Decay characteristics of HIV-1-infected compartments during combination therapy. *Nature*, 387, 188–191.
- Preskill, J. (2018). Quantum computing in the NISQ era and beyond. *Quantum*, 2, 79.
- Rice, J.R. (1976). The algorithm selection problem. *Advances in Computers*, 15, 65–118.
- Smith, W.E. (1956). Various optimizers for single-stage production. *Naval Research Logistics Quarterly*, 3(1–2), 59–66.
- Snoek, J., Larochelle, H., & Adams, R.P. (2012). Practical Bayesian optimization of machine learning algorithms. *NeurIPS 2012.*
- Srinivas, N., et al. (2010). Gaussian process optimization in the bandit setting. *ICML 2010.*
- Sutton, R.S., & Barto, A.G. (1998). Reinforcement Learning: An Introduction. MIT Press.
- Vanschoren, J. (2018). Meta-learning: A survey. *arXiv:1810.03548.*
- Vinyals, O., Fortunato, M., & Jaitly, N. (2015). Pointer networks. *NeurIPS 2015.*
- Wolpert, D.H., & Macready, W.G. (1997). No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1), 67–82.

---

1. A specification is *GS-derivable* if a stateless reader with no prior context — a fresh human contributor, or a new AI session — can apply any mutation specified in the genome's `-- GS EVOLUTION SPEC` blocks to source code by reading the specification alone, without consulting external state or institutional memory. Operationally, each `-- GS EVOLUTION SPEC` block declares a mutation type (e.g. `ParameterAdjust`, `StructuralRewire`), the target entity, the parameter or signal being modified, the delta or wiring change, the source file and symbol the mutation maps to, and the verification command (e.g. `cargo test --lib -- runtime`). The block is intentionally self-contained: it carries every piece of information needed to apply the mutation, audit it, and confirm it. See Ghiringhelli (2026, *Generative Specification*) for the formal definition and the broader GS discipline this property anchors.↩

2. A domain satisfies the **structural criterion for T5** when the *class* of optimal solution — not merely its parameters — changes during the experiment horizon, making `StructuralRewire` rather than `ParameterAdjust` the load-bearing primitive. The criterion is falsifiable: a domain marked Y can be demoted to T3/T4 if an empirical study shows its optimal-solution class is in fact stable; a domain marked N can be

promoted if structural drift is observed. The two calibration domains (`biosphere`, `ocean_circulation`) are marked N by design to demonstrate the framework does not default to T5 universally.↩